

`pgm1.c` is replaced with the line of code `int a = 5;`. Since the preprocessed file `pgm1.i` is very large, only the last six lines are printed with the command `tail -6 pgm1.i`.

Similarly, for the assembly code file also, only the last six lines are printed with the command `tail -6 pgm1.s`. The command `ls` shows all the files produced during the step by step compilation process. Finally, the figure shows the output by executing the absolute machine code in file `exfile`.

Options for C versions and warnings

The C programming language has the following versions -- K&R C, ANSI C (ISO C), c99, c11, and c18. Programmers can specify the version of C so that the gcc compiler can check whether the code is compliant with that particular standard or not. For example, the options `-ansi`, `-std=c99`, and `-std=c11` can be used to specify ANSI C, c99, and c11 versions of C, respectively. For a better understanding of this, consider the small C program `pgm2.c`.

```
#include<stdio.h>
int main()
{
    int unix;
    return 0;
}
```

Let us compile the C program `pgm2.c` with the commands `gcc pgm2.c` and `gcc -ansi pgm2.c`. Figure 3 shows the terminal after the commands have been executed. We get an error with the first command whereas the second command produces the executable file `a.out`. So why do we get an error with the first command? GCC provides predefined macros like UNIX as an extension, which are not part of standard C. So, the identifier `unix` is valid in ANSI C whereas in GCC it is not.

Now let us familiarise ourselves with a few options like `Wall`, `Werror`,

```
$ gcc pgm1.c
$ ./a.out
Square of 5 is 25

$ gcc pgm1.c -o exfile
$ ls
a.out exfile pgm1.c
$ ./a.out
Square of 5 is 25
```

Figure 1: Output of the program `pgm1.c`

```
$ gcc -E pgm1.c -o pgm1.i
$ tail -6 pgm1.i
int main()
{
    int a = 5;
    printf("Square of %d is %d\n", a, a*a);
    return 0;
}
$ gcc -S pgm1.i
$ tail -6 pgm1.s
    ret
    .cfl_endproc
.LFE0:
    .size    main, .-main
    .ident   "GCC: (Ubuntu 7.3.0-27ubuntu1-18.04) 7.3.0"
    .section .note.GNU-stack,"",@progbits
$ gcc -c pgm1.s
$ gcc pgm1.o -o exfile
$ ls
exfile pgm1.c pgm1.i pgm1.o pgm1.s
$ ./exfile
Square of 5 is 25
```

Figure 2: Step by step compilation of the C program `pgm1.c`

```
$ gcc pgm2.c
pgm2.c: In function 'main':
pgm2.c:4:9: error: expected identifier or '(' before numeric constant
    int unix;
        ^
$ gcc -ansi pgm2.c
$
```

Figure 3: Differences between GCC C and ANSI C

```
$ gcc -ansi -Wall pgm2.c
pgm2.c: In function 'main':
pgm2.c:4:9: warning: unused variable 'unix' [-Wunused-variable]
    int unix;
        ^
$ gcc -ansi -Wall -Werror pgm2.c
pgm2.c: In function 'main':
pgm2.c:4:9: error: unused variable 'unix' [-Werror=unused-variable]
    int unix;
        ^
cc1: all warnings being treated as errors
```

Figure 4: The options `-Wall` and `-Werror`

etc, which are used for customising warnings during compilation. Previously, when we have compiled the C program `pgm2.c`, we didn't get any warnings (see Figure 3). Now, let us compile `pgm2.c` with the option `-Wall` as `gcc -ansi -Wall pgm2.c`.

Figure 4 shows us that on compilation, we get the warning message *Unused variable* on the line of code `int unix;`. However, the option `-Wall` produces the executable file `a.out` whereas if the option `-Werror` is also added, no executable file is produced. This example also tells us that multiple

gcc options can be used for a single compilation. The options `-Wall` and `-Werror` help us write code that does not contain any potential bugs in the later stages of development. I believe these options, when enabled, will allow students of C programming to write good quality code.

A C program without `main()` function

The gcc compiler allows us even to compile a C program without the `main()` function. This is one of the several solutions to the popular C programming puzzle, "Write a C program without the `main()` function." This technique can also save at least three characters in C code golfing competitions, which are programming competitions in which the competitors try to write a program with the shortest source code possible to solve the given task. Consider the C program `pgm3.c` without the `main()` function, given below.

```
#include<stdio.h>
#include<stdlib.h>
int X( )
{
    printf("\nHello World\n");
    exit(0);
}
```

We must be careful to use the `exit(0)` function instead of `return 0;` statement to terminate the program because the `main()` function is absent in the code. Figure 5 shows the output of the program `pgm3.c` when compiled with the command `gcc -nostartfiles pgm3.c`. In the figure, we can see that the gcc compiler issues a warning stating that the `main()` function is absent.

Options for optimisation

There are a number of options provided by gcc for optimisation of